

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск подстроки в строке

Студент гр. 4345

Гаврилов Е.В.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Изучить и реализовать алгоритм Кнута-Морриса-Пратта. Применить его к решению задач поиска подстроки в строке и проверки, является ли одна строка циклическим сдвигом другой.

Задание

Часть 1. Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 25000$) и текста T ($|T| \leq 500000$) найдите все вхождения P в T .

Ввод: строка P (шаблон), строка T (текст).

Вывод: индексы начал вхождений P в T , разделенные запятой; если P не входит в T , то вывести -1.

Пример

Ввод

ab

abab

Вывод

0, 2

Часть 2. Заданы две строки A ($|A| \leq 5000000$) и B ($|B| \leq 5000000$). Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B , склеенного с префиксом B). Например, defabc является циклическим сдвигом abcdef.

Вход: первая строка A , вторая строка B

Выход: если A является циклическим сдвигом B , то индекс начала строки B в A ; иначе -1. Если возможно несколько сдвигов, вывести первый индекс.

Пример

Ввод

defabc

abcdef

Вывод

3

Выполнение работы

В ходе работы был реализован алгоритм Кнута-Морриса-Пратта, который по входным строкам `pattern`, `text` находит все индексы начал вхождений `pattern` в `text` за линейное время — функция `kmp(pattern, text string)`.

Этот алгоритм использует дополнительную информацию в виде префикс-функции строки `pattern`, для нахождения которой также реализован отдельный алгоритм в виде функции `computePrefixFunction(pattern string)`, возвращающей массив — префикс-функцию строки `pattern`.

Для решения конкретных задач реализованы функции `solveKmp()` и `solveCyclicShift()`, которые, используя алгоритм КМП, решают задачи соответственно: ввода паттерна и строки и нахождения списка индексов вхождений в строку, затем вывода этого списка; ввода-вывода и проверки, является ли первая вводимая строка циклическим сдвигом второй.

Описание функции `computePrefixFunction(pattern string) []int`

На вход подается строка `pattern`. Вычисляется ее длина в переменную `patternLength`. Выделяется и инициализируется нулями массив `prefix []int` длины `patternLength` под значения префикс-функции, который в итоге и будет возвращен.

Далее происходит процесс вычисления. Для переменных-индексов i, j , инициализированных как $i := 1, j := 0$ (для хранения индексов текущего рассматриваемого суффикса и префикса соответственно) начинается цикл с условием выхода $i \geq patternLength$:

- Если символы на позициях i, j (`pattern[i]` и `pattern[j]`) совпадают, то значение префикс-функции на позиции i (для текущего рассматриваемого суффикса) принимается равным $j+1$ (то есть, длине текущего рассматриваемого префикса), после этого переменные i, j каждая увеличивается на 1, и мы рассматриваем строку далее, начав новую итерацию цикла.

- При несовпадении символов мы рассматриваем две ситуации:

а. Если $j == 0$, то текущий префикс и так пуст, то есть, мы можем продолжить движение по строке, оставив значение префикс-функции для текущего символа строки нулевым, переменная i увеличивается на 1.

б. Иначе мы никуда не двигаемся по строке и вычисляем новый индекс префикса как значение префикс-функции от предыдущего символа префикса ($j = \text{prefix}[j-1]$), для того, чтобы не совершать лишнее движение по строке, ведь мы уже знаем, что некоторое количество предыдущих символов (а именно, ровно $\text{prefix}[j-1]$) совпадает, и, возможно, текущий суффикс все же является равным префиксу, начиная с другого символа.

По окончании обработки одной из этих двух ситуаций итерация цикла окончена, производится переход к следующей.

По завершении цикла функция возвращает полученный массив префикс-функций.

Описание функции `kmp(pattern, text string) []int`

На вход подаются строки *pattern*, *text*. Возвращается массив индексов *matches []int*, содержащий индексы вхождений *pattern* в *text*.

Для начала инициализируются нулями переменные *patternIndex* и *textIndex*, используемые далее для прохода по строке и искомой подстроке. Вычисляется и заносится в переменную *textLength* длина строки, а в переменную *patternLength* — длина шаблона. Объявляется массив *matches []int* для хранения результата.

Вычисляется и заносится в переменную *prefix* префикс-функция строки *pattern* с помощью функции *computePrefixFunction(pattern)*.

Далее выполняется основной цикл функции с условием выхода $\text{textIndex} \geq \text{textLength}$ (т.е. вся исходная строка обработана):

- Если символы на позициях *textIndex* и *patternIndex* совпадают ($\text{pattern}[\text{patternIndex}] == \text{text}[\text{textIndex}]$), то *textIndex* и *patternIndex* инкрементируются, и, если в результате операции индекс шаблона стал равным его длине ($\text{patternIndex} == \text{patternLength}$), то мы нашли вхождение, и в массив *matches* записывается индекс первого символа вхождения,

вычисляемый как $textIndex - patternIndex$, а значение $patternIndex$ пересчитывается как префикс от предыдущего символа (который мы только что рассмотрели): $patternIndex = prefix[patternIndex - 1]$, т. к. если паттерн циклический, то нам нужно рассматривать строку дальше с учетом того, что ровно $prefix[patternIndex - 1]$ предыдущих символов текста совпадало с искомым шаблоном. Итерация цикла окончена, производится переход к следующей.

- Если символы не совпали, то:

при условии $patternIndex == 0$ инкрементируется $textIndex$, т. к. мы находимся на начале шаблона, и символы не совпали

иначе мы не двигаемся по строке, а принимаем $patternIndex = prefix[patternIndex - 1]$, так как предыдущие символы исходной строки совпадали со строкой $pattern$ — $patternIndex - 1$ раз.

Итерация цикла окончена, производится переход к следующей.

По завершении выполнения цикла возвращается полученный массив $matches$ с индексами первых символов вхождений строки $pattern$ в $text$.

Описание функции solveKmp()

Ничего не принимается на вход, ничего не возвращается.

Объявляются переменные $pattern$, $text$, в них из стандартного ввода считываются две строки, разделенные пробельными символами. Массив $matches$ заполняется индексами начал вхождений $pattern$ в $text$, для чего вызывается функция $kmp(pattern, text)$. Далее в стандартный вывод печатаются разделенные запятыми значения из массива $matches$, или «-1», если он пуст.

Описание функции solveKmp()

Ничего не принимается на вход, ничего не возвращается.

Объявляются переменные $text$, $pattern$, в них из стандартного ввода считываются две строки, разделенные пробельными символами. Если их длины не совпадают, в стандартный вывод печатается «-1», иначе вызывается функция $kmp(pattern, text + text)$ для поиска образца в удвоенном тексте, чтобы

найти циклический сдвиг. Если она вернула пустой массив, печатается «-1», иначе печатается первый найденный сдвиг (с наименьшим индексом).

Сложность алгоритма

Сложность алгоритма Кнута-Морриса-Пратта, где входные данные это P — длина искомой подстроки, T — длина строки:

по времени: $O(P + T)$, поскольку за $O(P)$ создаем массив префикс-функций, а далее проходим основным циклом процедуры поиска подстроки за $O(T)$, сложением получаем итоговую асимптотику.

по памяти: $O(P + T)$, поскольку нужно создать и хранить массив префикс-функций, зависящий от размера исходной подстроки, для которой он создается — $O(P)$. Также создается массив для хранения выходных данных — индексов вхождений подстроки в строку, который в худшем случае составляет $O(T)$, поэтому оценка худшего случая — $O(P + T)$.

Исходный код программы см. в приложении А.

Результаты тестирования функции, вычисляющей все индексы вхождений подстроки в строку см. в Таблице 1.

Результаты тестирования функции, проверяющей по входным строкам A и B , является ли A циклическим сдвигом B см. в Таблице 2.

Таблица 1 – Результаты тестирования КМП

№	Ввод	Вывод	Комментарий
1	abrakadabra abrakadabrakadabrakadabra	0,7,14	Наложение подстрок
2	koshka lukoshkalukoshka	2,10	Тривиальный случай
3	go johny	-1	Нет искомой подстроки
4	gogo gogo	0	Строка = шаблон
5	gogogo gogo	-1	Шаблон больше строки

Таблица 2 – Результаты тестирования нахождения циклического сдвига

№	Ввод	Вывод	Комментарий
1	defabc abcdef	3	Корректный сдвиг
2	abc bdef	-1	Разные длины строк
3	abd efc	-1	Разные строки
4	ababab bababa	1	Строка может быть получена разными сдвигами

Вывод

Реализован алгоритм Кнута-Морриса-Пратта. Дана асимптотическая оценка сложности его реализации по времени: $O(P + T)$, памяти: $O(P + T)$, где P — длина искомого шаблона, T — длина строки, в которой осуществляется его поиск. Описано применение данного алгоритма к конкретным задачам: поиска всех индексов начал вхождений шаблона в строку и проверки, является ли одна строка циклическим сдвигом другой.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.go

```
package main

import (
    "bufio"
    "fmt"
    "os"
)

func computePrefixFunction(pattern string) []int {
    fmt.Printf("Computing prefix function for pattern: %s\n",
pattern)
    patternLength := len(pattern)
    prefix := make([]int, patternLength)
    i := 1
    j := 0
    for i < patternLength {
        fmt.Printf("Current state: i=%d, j=%d; pattern[i]=%c,
pattern[j]=%c\n", i, j, pattern[i], pattern[j])
        if pattern[i] == pattern[j] {
            prefix[i] = j + 1
            fmt.Printf("Symbols match; prefix[i]=%d; i and j
shifted right\n", prefix[i])
            i++
            j++
        } else {
            if j == 0 {
                fmt.Println("Symbols do not match; prefix is in
the beginning; i shifted right")
                i++
            } else {
                fmt.Println("Symbols do not match; j shifted to
prefix[j-1]")
                j = prefix[j-1]
            }
        }
    }
    fmt.Printf("Final prefix array for pattern %s: %v\n\n", pattern,
prefix)
    return prefix
}

func kmp(pattern, text string) []int {
    patternIndex := 0
    textIndex := 0
    textLength := len(text)

    var matches []int
    prefix := computePrefixFunction(pattern)
    fmt.Printf("Searching for pattern '%s' in text '%s' using KMP
algorithm\n", pattern, text)
    for textIndex < textLength {
```

```

        fmt.Printf("\tNow on textIndex=%d, patternIndex=%d\n",
textIndex, patternIndex)
        fmt.Printf("Current symbols: on text '%c', pattern '%c'\n",
text[textIndex], pattern[patternIndex])
        if pattern[patternIndex] == text[textIndex] {
            fmt.Println("Symbols match; shift right on both
strings")
            patternIndex++
            textIndex++
            if patternIndex == len(pattern) {
                matches = append(matches, textIndex-patternIndex)
                fmt.Printf("What's more, full substring match is
found! Index added to matches: %v\n", matches)
                patternIndex = prefix[patternIndex-1]
                fmt.Printf("patternIndex shifted left to the
position of last prefix function: %d\n", patternIndex)
            }
        } else {
            fmt.Print("Symbols do not match; ")
            if patternIndex == 0 {
                fmt.Println("we're on the beginning of the
pattern, so shift right on a text string")
                textIndex++
            } else {
                fmt.Println("we're not on the beginning of the
pattern, so we change")
                fmt.Println("pattern position to the position
pointed by last prefix function")
                patternIndex = prefix[patternIndex-1]
            }
        }
    }
    fmt.Println("\tReached end of text")
    fmt.Printf("Final array consisting of starting indices in text
where pattern occurs: %v\n\n", matches)
    return matches
}

func solveCyclicShift() {
    var pattern, text string
    reader := bufio.NewReader(os.Stdin)
    fmt.Fscan(reader, &text)
    fmt.Fscan(reader, &pattern)
    if len(pattern) != len(text) {
        fmt.Println("Pattern and text's lengths are not equal")
        fmt.Println("-1")
    } else {
        matches := kmp(pattern, text+text)
        if len(matches) > 0 {
            fmt.Println(matches[0])
        } else {
            fmt.Println("Matches not found")
            fmt.Println("-1")
        }
    }
}

func solveKmp() {

```

```

var pattern, text string
reader := bufio.NewReader(os.Stdin)
fmt.Fscan(reader, &pattern)
fmt.Fscan(reader, &text)
matches := kmp(pattern, text)
if len(matches) > 0 {
    for i, match := range matches {
        if i > 0 {
            fmt.Print(",")
        }
        fmt.Print(match)
    }
    fmt.Println()
} else {
    fmt.Println("Matches not found")
    fmt.Println("-1")
}
}

func main() {
    solveKmp()
    solveCyclicShift()
}

```